ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

TIỂU LUẬN MÔN HỌC
IT5440 – NGUYÊN LÝ VÀ KỸ THUẬT PHÂN TÍCH CHƯƠNG TRÌNH
(Principle and Technique of Program Analysis)

Đề tài 02-KH – Phát hiện lỗi và định vị lỗi (hướng nghiên cứu khoa học)

GROUNDED OR GUESSED: TÁI HIỆN VÀ ĐÁNH GIÁ VIỆC ĐỊNH VỊ LỖI NGỮ NGHĨA BẰNG MÔ HÌNH NGÔN NGỮ LỚN

Báo cáo tiến độ lần 2 (18–19/11/2026)

Giảng viên hướng dẫn: PGS. TS. Huỳnh Quyết Thắng

Nhóm M9: Doãn Văn An – 20252778M (nhóm trưởng)
Tạ Công Nam – 20252640M
Nguyễn Hoàng Hải – 20252583M
Đỗ Xuân Tráng – 20252774M
Hoàng Trung Kiên – 20251333M
Mai Văn Tùng – 20252788M
Nguyễn Đức Tùng – 20252772M

Học kỳ: 20261

Mã nguồn và dữ liệu: `https://github.com/doanan293/grounded-or-guessed`

HÀ NỘI – 2026

Cam kết về tính trung thực

Nhóm M9 cam kết nội dung tiểu luận là kết quả làm việc của nhóm dưới sự hướng dẫn của PGS. TS. Huỳnh Quyết Thắng. Mọi số liệu trong báo cáo được sinh tự động từ mã nguồn và dữ liệu công khai tại `https://github.com/doanan293/grounded-or-guessed`; mỗi con số trong văn bản là một macro được tạo từ tệp dữ liệu tương ứng, không gõ tay.

Phương pháp SemLoc, mã nguồn gốc và bộ dữ liệu SemFault-250 thuộc về nhóm tác giả của bài báo gốc [1] và được dùng theo giấy phép Apache-2.0; mọi phần lấy từ SemLoc đều được ghi nguồn và không được trình bày như đóng góp của nhóm. Hai bài báo còn lại do giảng viên giao [2, 3] và các tài liệu khác được trích dẫn tại chỗ sử dụng.

Theo hướng dẫn của môn học, nhóm có sử dụng công cụ trí tuệ nhân tạo để hỗ trợ đọc tài liệu, viết mã nguồn, chạy thực nghiệm và soạn thảo; mọi kết quả đã được nhóm kiểm tra lại bằng mã nguồn và dữ liệu đi kèm.

Hà Nội, ngày tháng 11 năm 2026

Đại diện nhóm M9 (nhóm trưởng)

Doãn Văn An

Thông tin nhóm và phân công công việc

THÀNH VIÊN

Nhóm M9 gồm bảy học viên, đã chốt danh sách và đề tài 02-KH (phát hiện lỗi và định vị lỗi, hướng nghiên cứu khoa học). Nhóm trưởng: **Doãn Văn An** (20252778M), đầu mối liên hệ với giảng viên và người giữ kho mã của nhóm.

Bảng 1: Thành viên nhóm M9

STT	Họ và tên	MSHV	Vai trò chính
1	Doãn Văn An	20252778M	Nhóm trưởng; điều phối, hạ tầng thực nghiệm, phương pháp và viết bài
2	Tạ Công Nam	20252640M	Bộ nhãn và duyệt nhãn; phân tích kết quả
3	Nguyễn Hoàng Hải	20252583M	Tổng quan tài liệu và bộ từ khóa
4	Đỗ Xuân Tráng	20252774M	Lỗi thật BugsInPy (RQ3)
5	Hoàng Trung Kiên	20251333M	Biến thể đổi tên và các họ mô hình (RQ3)
6	Mai Văn Tùng	20252788M	Slide
7	Nguyễn Đức Tùng	20252772M	Công cụ phân tích tĩnh; kiểm tra số liệu

PHÂN CÔNG THEO MẢNG VIỆC

Công việc của cả tiểu luận được chia thành bảy mảng lớn. Nhóm trưởng phụ trách phần lớn khối lượng (hạ tầng, tái hiện, phương pháp đề xuất và viết bài); mỗi thành viên còn lại phụ trách chính một mảng và cùng tham gia duyệt nhãn.

Bảng 2: Phân công công việc theo mảng

Mảng việc	Phụ trách chính	Cùng thực hiện
Tổng quan tài liệu, bộ từ khóa, phân tích ba bài báo được giao	Nguyễn Hoàng Hải	Doãn Văn An
Hạ tầng gọi LLM, tái hiện SemLoc và kiểm tra cách đánh giá (RQ1)	Doãn Văn An	Tạ Công Nam
Bộ nhãn: kiểm chứng bằng thực thi, rà soát nhãn, duyệt mù	Tạ Công Nam	Doãn Văn An; cả nhóm duyệt nhãn
Phương pháp đề xuất và so sánh (RQ2): prompt, thí nghiệm loại bỏ, các cách bổ sung, phân tích tĩnh	Doãn Văn An	Nguyễn Đức Tùng
Độ bền (RQ3): lỗi thật BugsInPy, mã đổi tên, các họ mô hình	Đỗ Xuân Tráng, Hoàng Trung Kiên	Doãn Văn An
Phân tích thất bại, đóng góp, triết lý khoa học, mối đe dọa	Doãn Văn An	Tạ Công Nam, Hoàng Trung Kiên
Báo cáo, bài báo arXiv, slide, gói tái hiện	Doãn Văn An	Mai Văn Tùng, Nguyễn Đức Tùng

CÔNG CỤ AI SỬ DỤNG

- **Claude Code** (Anthropic) là trợ lý chính: đọc và tóm tắt tài liệu, viết mã thực nghiệm và kiểm thử tự động, chạy thực nghiệm, soạn báo cáo LaTeX và slide. Mọi thay đổi được nhóm trưởng duyệt trước khi đưa vào kho mã.
- **LLM là đối tượng thực nghiệm**, gọi qua một proxy tương thích OpenAI: Gemini 3.8 Flash là mô hình chính; Claude Sonnet 4.6 (mô hình của bài báo gốc) để đối chiếu khi tái hiện; cho RQ3 thêm Gemini 3.5 Flash-Lite, Claude Sonnet 5.5, GPT-6 luna và gpt-oss-120b (mô hình mở của OpenAI).

Để kết quả kiểm chứng được dù có dùng AI, mọi con số trong báo cáo là macro sinh

tự động từ dữ liệu, mọi lượt gọi LLM được ghi nhật ký và lưu bộ đệm để chạy lại, và kho mã có bộ kiểm thử tự động.

Mục lục

Cam kết về tính trung thực i

Thông tin nhóm và phân công công việc ii

Danh mục từ viết tắt và thuật ngữ x

1 Mở đầu 1

1.1 Đặt vấn đề 1

1.2 Câu hỏi nghiên cứu 2

1.3 Kết quả đến thời điểm báo cáo 2

1.4 Cấu trúc báo cáo 3

2 Tổng quan tài liệu và từ khóa 4

2.1 Phương pháp tìm và chọn tài liệu 4

2.1.1 Nguồn và tiêu chí chọn 4

2.1.2 Bộ từ khóa 4

2.2 Ba bài báo được giao 5

2.2.1 SemLoc: neo lập luận của LLM vào ràng buộc thực thi được . . . 5

2.2.2 EffiHolmes: định vị chỗ sửa lỗi hiệu năng bằng so sánh hai lần chạy 6

2.2.3 Khai thác báo cáo sự cố kết hợp LLM trong công nghiệp 7

2.3 Các hướng nghiên cứu liên quan 7

2.3.1 Định vị lỗi dựa trên phổ và đột biến 7

2.3.2 Định vị lỗi bằng mô hình ngôn ngữ lớn 8

2.3.3 Thực tế phát triển phần mềm 8

2.3.4 Bộ dữ liệu lỗi và tính hợp lệ của đánh giá 9

2.3.5 Bản sửa vượt kiểm thử nhưng không đúng 9

2.3.6 Nhiễm dữ liệu và tái hiện khi dùng LLM 9

2.4 So sánh và khoảng trống nghiên cứu 10
2.4.1 Bảng so sánh 10
2.4.2 Khoảng trống và hướng của nhóm 11

3 Cơ sở lý thuyết 12
3.1 Định vị lỗi dựa trên phổ độ phủ 12
3.1.1 Ma trận độ phủ và các công thức nghi ngờ 12
3.1.2 Chỉ số đánh giá và cách xử lý hòa điểm 12
3.2 Lỗi ngữ nghĩa 13
3.2.1 Vì sao độ phủ mất tín hiệu 13
3.2.2 Định nghĩa khách quan bằng độ phủ 13
3.3 SemLoc: phổ vi phạm ràng buộc ngữ nghĩa 14
3.3.1 Quy trình bốn bước 14
3.3.2 Điểm thưởng cho dòng LLM tự đoán 14
3.4 Phân tích tĩnh và phân tích động 15
3.5 Công cụ thống kê 15
3.5.1 Kiểm định cặp 15
3.5.2 Khoảng tin cậy bootstrap theo cụm và giá trị Shapley 16

4 Phương pháp và thiết kế thực nghiệm 17
4.1 Câu hỏi nghiên cứu và giả thuyết 17
4.1.1 Ba câu hỏi nghiên cứu 17
4.1.2 Giả thuyết và tiêu chí bác bỏ 17
4.2 Dữ liệu 18
4.2.1 SemFault-250 và cách chia tập 18
4.2.2 SemFault-250 đã sửa: kiểm chứng dòng lỗi chuẩn bằng thực thi . 18
4.2.3 Lỗi thật BugsInPy 19
4.2.4 Biến thể đổi tên định danh 19

4.3 Các phương pháp so sánh 19
4.3.1 Các phương pháp có sẵn 19
4.3.2 LLM + kiểm thử thất bại: phương pháp đề xuất 20
4.4 Kiểm tra cách đánh giá và phân rã đóng góp 20
4.4.1 Kiểm tra cách đánh giá của SemLoc 20
4.4.2 Phân rã "grounded hay guessed" 21
4.5 Thiết lập thực nghiệm 21
4.5.1 Mô hình và hạ tầng 21
4.5.2 Lặp, kiểm tra sức khỏe lượt chạy và thống kê 21

5 Kết quả thực nghiệm 23
5.1 RQ1 – Tái hiện và kiểm tra cách đánh giá 23
5.1.1 Tái hiện và hai cách chấm điểm 23
5.1.2 Kiểm chứng phản thực tế và lỗi của gói mã công bố 24
5.1.3 Ảnh hưởng của dòng lỗi chuẩn 25
5.2 RQ2 – LLM thật sự cần gì? 25
5.2.1 Một prompt có kiểm thử thất bại so với quy trình nhiều bước . . 25
5.2.2 Độ nhạy với quy ước gán nhãn 26
5.2.3 Grounded hay guessed: giá trị biên và phân rã 27
5.3 RQ3 – Độ bền 28
5.4 Tóm tắt trả lời các câu hỏi nghiên cứu 28

6 Kết luận và hướng phát triển 29
6.1 Kết luận 29
6.2 Hướng phát triển 29

Danh sách bảng

1 Thành viên nhóm M9 ii

2 Phân công công việc theo mảng iii

2.1 Bộ từ khóa của đề tài 5

2.2 So sánh ba bài báo được giao với hướng của nhóm 10

5.1 Kết quả công bố tính lại theo hai cách chấm (nhật ký của tác giả, 250 chương trình) 24

5.2 Kết quả chính trên bộ nhãn gốc và bộ đã sửa (tập test, Gemini 3.8 Flash, trung bình 3 lần chạy) và chi phí token mỗi chương trình 25

5.3 Acc@1 khớp đúng dòng, nới lỏng (dòng điều kiện – dòng đầu của thân) và cận trên (tập test, Gemini 3.8 Flash, trung bình 3 lần chạy) 26

5.4 Phân rã Shapley độ chính xác của SemLoc (lượt chạy của nhóm, 3 lần) . 27

5.5 Trả lời các câu hỏi nghiên cứu đến thời điểm báo cáo 28

Danh sách hình vẽ

3.1 Quy trình của SemLoc (theo [1]); khung nét đứt là thành phần nhóm phát hiện trong mã nguồn 14

Danh mục từ viết tắt và thuật ngữ

Viết tắt / thuật ngữ	Ý nghĩa
Bộ phán định	Quy trình tự động quyết định một dòng có phải dòng lỗi hay không: một bản sửa nhỏ trên dòng đó làm mọi kiểm thử đạt (oracle)
Bootstrap	Lấy mẫu lại có hoàn lại để ước lượng khoảng tin cậy; ở đây lấy mẫu theo chương trình, 10000 lần
Grounded / guessed	Phần thứ hạng do bằng chứng thực thi quyết định / phần do LLM tự đoán
Bản sửa hợp lệ bề ngoài	Bản sửa làm mọi kiểm thử đạt nhưng không sửa đúng chỗ lỗi (plausible patch)
CI	Tích hợp liên tục: hệ thống tự chạy kiểm thử khi mã được đẩy lên kho
Phân tích tĩnh / động	Phân tích mã không chạy chương trình / dựa trên việc chạy chương trình
TDD	Phát triển hướng kiểm thử: viết kiểm thử trước khi viết mã
Phương pháp đối chứng	Phương pháp đơn giản dùng để so sánh (baseline)
Prompt (lời nhắc)	Văn bản đầu vào gửi cho LLM
Quy trình nhiều bước	Chuỗi các bước LLM và thực thi nối tiếp nhau (pipeline), như SemLoc
Token	Đơn vị văn bản mà LLM xử lý và tính phí
Acc@k	Tỉ lệ chương trình có dòng lỗi nằm trong k dòng bị nghi ngờ nhất (độ chính xác top-k)
CF	Kiểm chứng phản thực tế (counterfactual verification): vá thử một dòng rồi chạy lại kiểm thử
CI	Khoảng tin cậy (confidence interval), ở đây là khoảng 95% theo bootstrap
CovGate	Cách bổ sung do nhóm thử: kết hợp độ phủ với điểm của SemLoc
DD	Gỡ lỗi chia đôi (delta debugging)
GT	Dòng lỗi chuẩn (ground truth) của bộ dữ liệu
LLM	Mô hình ngôn ngữ lớn (large language model)
RQ	Câu hỏi nghiên cứu (research question)

Viết tắt / thuật ngữ	Ý nghĩa
SBFL	Định vị lỗi dựa trên phổ độ phủ (spectrum-based fault localization)
SemFault-250	Bộ dữ liệu 250 chương trình Python có lỗi ngữ nghĩa của SemLoc
SemFault-250 đã sửa	Bộ dữ liệu trên với dòng lỗi chuẩn được nhóm kiểm chứng và sửa lại bằng thực thi (tiếng Anh: SemFault-250-repaired)
Shapley	Giá trị Shapley: cách chia một tổng cho các thành phần theo đóng góp biên trung bình

Chương 1. Mở đầu

1.1 ĐẶT VẤN ĐỀ

Định vị lỗi là bước tốn công nhất của gỡ lỗi: khi chạy kiểm thử thấy thất bại — trên máy cá nhân hay trên hệ thống tích hợp liên tục (CI) — lập trình viên phải tìm ra dòng mã gây lỗi trong hàng trăm dòng có liên quan. Các công cụ phân tích tĩnh thông dụng như trình kiểm tra phong cách mã (lint) hay trình kiểm tra kiểu không giúp được ở bước này khi lỗi nằm ở logic: mã vẫn đúng cú pháp, đúng kiểu, chỉ cho ra kết quả sai. Thông tin duy nhất chỉ ra lỗi là chính các kiểm thử thất bại — và với cách phát triển hướng kiểm thử (TDD) [4], các kiểm thử đó được viết ra trước, mô tả ý định của mã.

Trợ lý AI đã trở thành công cụ hằng ngày của lập trình viên: các khảo sát lớn năm 2025 cho thấy phần lớn lập trình viên đang dùng hoặc định dùng chúng [5, 6], và gỡ lỗi là một trong các việc họ thường đem hỏi [7]. Nghiên cứu cũng cho thấy đưa thông tin lỗi cho LLM giúp nó định vị lỗi tốt hơn hẳn [8]. Từ đó, một cách làm đơn giản — chạy kiểm thử, đưa kết quả thất bại cho LLM, hỏi dòng nào sai — nhiều khả năng đã gần với cách nhiều lập trình viên đang làm.

Họ phương pháp kinh điển là định vị lỗi dựa trên phổ độ phủ (SBFL) [9, 10, 11]: một dòng càng hay được các kiểm thử thất bại chạy qua và càng ít được các kiểm thử thành công chạy qua thì càng đáng nghi. Với *lỗi ngữ nghĩa* — chương trình chạy đúng đường nhưng tính sai giá trị — hai loại kiểm thử thường chạy qua cùng một tập dòng, và SBFL gần như mất tín hiệu.

SemLoc [1] — bài báo trung tâm trong ba bài được giảng viên giao — đề xuất một quy trình nhiều bước cho lỗi ngữ nghĩa: LLM viết ra các ràng buộc ngữ nghĩa thực thi được, ràng buộc được chèn vào chương trình, kiểm thử được chạy để lấy phổ vi phạm, rồi LLM vá thử các dòng nghi ngờ để kiểm chứng. Bài báo tuyên bố độ chính xác đến từ việc *neo lập luận của LLM vào bằng chứng thực thi* (structured grounding) và báo cáo

Acc@1 42,8% trên bộ dữ liệu SemFault-250. Câu hỏi tự nhiên với người làm phần mềm là: **quy trình phức tạp đó có thật sự cần không, khi so với cách làm đơn giản đó?**

1.2 CÂU HỎI NGHIÊN CỨU

Tiểu luận đặt câu hỏi "grounded hay guessed?" cho SemLoc — độ chính xác đến từ bằng chứng thực thi hay từ việc LLM đoán — và tách thành ba câu hỏi nghiên cứu:

- **RQ1 – Tái hiện và kiểm tra cách đánh giá.** Kết quả công bố của SemLoc có tái hiện được không, và còn đứng vững khi chấm điểm công bằng, khi bỏ các thành phần không được mô tả trong bài báo, và khi kiểm tra lại chính các dòng lỗi chuẩn (nhãn) của bộ dữ liệu?
- **RQ2 – LLM thật sự cần gì?** Một prompt duy nhất có kiểm thử thất bại so với quy trình nhiều bước của SemLoc thì sao, về độ chính xác và chi phí? Thông tin nào trong prompt tạo ra khác biệt (kiểm thử thất bại hay độ phủ từng dòng)? Các cách bổ sung — gắn độ phủ vào SemLoc (CovGate), biểu quyết nhiều câu trả lời — và các công cụ phân tích tĩnh có thêm được gì không?
- **RQ3 – Độ bền.** Các kết luận có giữ được trên lỗi thật (BugsInPy [12]), trên mã đã đổi tên định danh, trên năm họ mô hình, và trên các bộ nhãn khác nhau không?

1.3 KẾT QUẢ ĐẾN THỜI ĐIỂM BÁO CÁO

Mỗi kết quả dưới đây gắn với số liệu được sinh tự động từ mã nguồn công khai.

1. **Hạng 1 của SemLoc là dòng LLM đoán.** Mã nguồn công bố cộng cho dòng LLM tự đoán một điểm thưởng lớn hơn mọi điểm ràng buộc — chi tiết không có trong bài báo — nên dòng đầu tiên của SemLoc *luôn* là dòng đoán. Bỏ điểm thưởng, Acc@1 đổi −9,1 điểm trên bộ nhãn đã sửa. Bước vá thử chiếm 42% lượt gọi LLM nhưng không đổi thứ hạng nào.

2. **Nhãn của bộ dữ liệu có vấn đề.** 56 nhãn gốc nằm trên dòng trống và 65 chương trình không có kiểm thử thất bại nào chạy tới dòng lỗi chuẩn. Nhóm dựng bộ nhãn *đã sửa* bằng thực thi: 82 nhãn được xác nhận, 46 nhãn được dời vị trí.
3. **Một prompt có kiểm thử thất bại thắng quy trình nhiều bước.** Phương pháp do nhóm đề xuất — một lượt gọi LLM với mã nguồn, tên kiểm thử thất bại và thông báo lỗi — hơn SemLoc +10,2 điểm Acc@1 và +14,5 điểm Acc@3 trên nhãn đã sửa, thắng ở Acc@1 trên cả bộ nhãn gốc lẫn bộ đã sửa, và tốn 2,0 nghìn token mỗi chương trình so với 14,0 nghìn của SemLoc.

Các phần đang thực hiện cho lần báo cáo 3: rà soát toàn bộ nhãn của bộ dữ liệu; thí nghiệm loại bỏ để biết thông tin nào trong prompt tạo ra khác biệt; các cách bổ sung và công cụ phân tích tĩnh; câu hỏi RQ3 (lỗi thật, mã đổi tên, các họ mô hình); đóng góp và triết lý khoa học.

1.4 CẤU TRÚC BÁO CÁO

Chương 2 tổng quan tài liệu và bộ từ khóa, trong đó phân tích ba bài báo được giao. Chương 3 trình bày cơ sở lý thuyết về SBFL, lỗi ngữ nghĩa, SemLoc, phân tích tĩnh và động, và các công cụ thống kê. Chương 4 mô tả câu hỏi nghiên cứu, dữ liệu và các bộ nhãn, các phương pháp so sánh và thiết kế thực nghiệm. Chương 5 trả lời ba câu hỏi nghiên cứu. Chương 6 kết luận và nêu hướng phát triển. Mã nguồn, dữ liệu và nhật ký gọi LLM được công bố tại `https://github.com/doanan293/grounded-or-guessed`.

Chương 2. Tổng quan tài liệu và từ khóa

2.1 PHƯƠNG PHÁP TÌM VÀ CHỌN TÀI LIỆU

2.1.1 Nguồn và tiêu chí chọn

Điểm xuất phát là ba bài báo giảng viên giao cho chương 10 “Phát hiện lỗi và định vị lỗi”: SemLoc [1], EffiHolmes [2] và báo cáo công nghiệp về khai thác báo cáo sự cố kết hợp LLM [3]. Từ ba bài này, nhóm mở rộng theo hai chiều: lần ngược các tài liệu được trích dẫn (định vị lỗi dựa trên phổ, dựa trên đột biến, gỡ lỗi chia đôi, các bộ dữ liệu lỗi) và tìm xuôi các nghiên cứu mới về định vị lỗi bằng LLM và về tính hợp lệ khi đánh giá LLM. Nguồn tra cứu là arXiv, ACM Digital Library, IEEE Xplore và Google Scholar; mỗi tài liệu được đối chiếu lại qua DOI hoặc mã arXiv trước khi đưa vào danh mục tham khảo.

Một tài liệu được chọn khi thỏa ít nhất một tiêu chí: (i) là phương pháp mà SemLoc so sánh hoặc kế thừa; (ii) là bộ dữ liệu lỗi dùng được cho Python; (iii) bàn về cách đánh giá định vị lỗi (hòa điểm, lỗi thiếu mã, lỗi nhân tạo so với lỗi thật); (iv) bàn về rủi ro khi dùng LLM trong nghiên cứu phần mềm (nhiễm dữ liệu, tái hiện); (v) cho biết lập trình viên thực sự dùng LLM để gỡ lỗi ra sao, hoặc bàn về nhãn sai và bản sửa “lách” kiểm thử. Kết quả là ba bài được giao và ba mươi mốt tài liệu bổ sung, tổng hợp ở Bảng 2.2.

2.1.2 Bộ từ khóa

Bảng 2.1 là bộ từ khóa song ngữ dùng để tìm kiếm và thống nhất thuật ngữ trong báo cáo.

Bảng 2.1: Bộ từ khóa của đề tài

Tiếng Anh	Tiếng Việt
fault localization	định vị lỗi
spectrum-based fault localization (SBFL)	định vị lỗi dựa trên phổ độ phủ
semantic fault	lỗi ngữ nghĩa
semantic constraint / invariant	ràng buộc ngữ nghĩa / bất biến
semantic violation spectrum	phổ vi phạm ràng buộc ngữ nghĩa
counterfactual verification	kiểm chứng phản thực tế
large language model (LLM)	mô hình ngôn ngữ lớn
structured grounding	neo lập luận vào biểu diễn có cấu trúc
program instrumentation	chèn mã kiểm tra vào chương trình
reproducibility	tính tái hiện được
data contamination / memorization	nhiễm dữ liệu huấn luyện / ghi nhớ
omission fault	lỗi thiếu mã
test-driven development (TDD)	phát triển hướng kiểm thử
plausible patch / test overfitting	bản sửa hợp lệ bề ngoài / quá khớp kiểm thử
label error (benchmark)	nhãn sai trong bộ dữ liệu
static / dynamic analysis	phân tích tĩnh / phân tích động

2.2 BA BÀI BÁO ĐƯỢC GIAO

2.2.1 SemLoc: neo lập luận của LLM vào ràng buộc thực thi được

Vấn đề. Với lỗi ngữ nghĩa, kiểm thử thành công và thất bại chạy qua cùng các câu lệnh, nên phổ độ phủ "sụp" và SBFL mất tín hiệu; còn LLM tự do thì ngẫu nhiên, không kiểm chứng được và khó tách nguyên nhân gốc với triệu chứng.

Phương pháp. SemLoc [1] gồm bốn bước: (1) tìm các *điểm neo* có cấu trúc trong chương trình (đầu hàm, điểm định nghĩa – sử dụng, biên vòng lặp, lệnh trả về); (2) một

lượt gọi LLM sinh các ràng buộc ngữ nghĩa theo một ngôn ngữ trung gian khép kín, mỗi ràng buộc gắn với một điểm neo và một biểu thức kiểm tra; (3) chèn các ràng buộc vào chương trình, chạy bộ kiểm thử, dựng ma trận "ràng buộc × kiểm thử" và chấm từng ràng buộc bằng công thức Ochiai, rồi truyền điểm về câu lệnh thiết lập tính chất đó; (4) kiểm chứng phản thực tế: với mỗi ràng buộc còn lại, LLM viết một bản vá một dòng tại dòng được quy cho ràng buộc, chạy lại kiểm thử và phân loại ràng buộc theo kết quả.

Đánh giá. Bộ dữ liệu SemFault-250 gồm 250 chương trình Python một hàm, một lỗi, mỗi chương trình có khoảng mười kiểm thử. SemLoc được so với SBFL (Ochiai, Tarantula), gỡ lỗi chia đôi và LLM hỏi thẳng, dùng Claude Sonnet 4.6 và Gemini 2.5 Pro ở ba mức nhiệt độ. Kết quả công bố: Acc@1 42,8%, Acc@3 68,0%; trên lỗi thật, bài báo chỉ có bảng cấp dòng cho bảy lỗi youtube-dl và tự nhận đây là kết quả khả thi.

Điểm cần kiểm chứng. Khi đọc mã nguồn công khai, nhóm thấy: bằng chứng trên lỗi thật rất ít; chương trình của SemFault-250 có văn phong đồng nhất như mã do LLM viết, nên khả năng LLM đã "thấy" dữ liệu chưa được kiểm tra; SemLoc bỏ hẳn tín hiệu độ phủ; mã chấm điểm phá hòa theo thứ tự dòng, tính mọi lần trúng của gỡ lỗi chia đôi là hạng 1 và loại các chương trình chạy lỗi khỏi mẫu số; mỗi cấu hình chỉ chạy một lần, không có kiểm định thống kê. Đây là đầu vào trực tiếp cho ba câu hỏi nghiên cứu.

2.2.2 EffiHolmes: định vị chỗ sửa lỗi hiệu năng bằng so sánh hai lần chạy

EffiHolmes [2] giải bài toán khó hơn ở một khía cạnh: lỗi hiệu năng không làm kiểm thử thất bại và không có vết ngăn xếp, nên các kỹ thuật định vị lỗi kinh điển không áp dụng được. Phương pháp chạy chương trình dưới hai mức tải (mặc định và phóng to), coi các hàm có chi phí tăng theo tải là điểm nóng, trích đường gọi gọn từ điểm vào tới điểm nóng, rồi để LLM lập luận có định hướng trên đường gọi đó. Trên 140 vấn đề hiệu năng thật từ năm kho mã khoa học dữ liệu, EffiHolmes vượt các phương pháp dựa trên truy hồi, tác tử và phân tích hiệu năng đơn thuần, và bền khi dùng mô hình nhỏ.

Bài học cho đề tài: cùng một ý tưởng với SemLoc — *đối chiếu hai lần thực thi để*

lọc nhiễu trước khi hỏi LLM — nhưng tín hiệu đối chiếu là chi phí theo tải thay vì kiểm thử đạt/không đạt. Cấu trúc câu hỏi nghiên cứu của bài (hiệu quả, loại bỏ thành phần, độ bền theo cỡ mô hình, phân tích thất bại) được nhóm dùng làm khuôn cho ba câu hỏi của mình. Lỗi hiệu năng nằm ngoài phạm vi thực nghiệm.

2.2.3 Khai thác báo cáo sự cố kết hợp LLM trong công nghiệp

Medeiros và cộng sự [3] khảo sát hai hệ thống Java lớn trong công nghiệp nhận hàng nghìn báo cáo sự cố mỗi tuần. Họ gom cụm vết ngăn xếp, xếp hạng tệp và phương thức đáng nghi, rồi đưa các xếp hạng này vào prompt của LLM; đánh giá hồi cứu năm LLM với bốn cấu hình prompt trên 38 lỗi sự cố thật, có kiểm tra thủ công. Cấu hình tốt nhất định vị được tới 71 trên 100 lỗi và sửa đúng 52 trên 100; thêm bảng xếp hạng phương thức làm tăng tỉ lệ câu trả lời nhắc tới phương thức đã được sửa.

Bài học cho đề tài: (i) *đưa bằng chứng chương trình đã khai thác (xếp hạng) vào prompt* giúp LLM định vị tốt hơn — đây chính là ý tưởng của phương pháp đối chứng LLM + độ phủ mà nhóm xây dựng; (ii) đầu ra của LLM cần được kiểm tra vì hay bị cắt cụt, lệch định dạng hoặc khẳng định không kiểm chứng. Bài cần dữ liệu công nghiệp nên không được dùng để thực nghiệm.

2.3 CÁC HƯỚNG NGHIÊN CỨU LIÊN QUAN

2.3.1 Định vị lỗi dựa trên phổ và đột biến

SBFL bắt đầu với Tarantula [9]; Ochiai [10] và DStar [13] là các công thức mạnh hơn được dùng làm mốc so sánh trong hầu hết nghiên cứu, và là nền của phổ vi phạm trong SemLoc. Các khảo sát của Wong và cộng sự [11] và de Souza và cộng sự [14] chỉ ra hai điểm yếu kinh điển của đánh giá SBFL: xử lý hòa điểm và lỗi thiếu mã (không có dòng nào để đổ lỗi). Định vị lỗi dựa trên đột biến (Metallaxis [15], MUSE [16]) thay độ phủ

bằng hành vi của các đột biến; gỡ lỗi chia đôi [17] và chuỗi nhân quả [18] là nền khái niệm cho bước kiểm chứng phản thực tế của SemLoc. Daikon [19] suy ra bất biến từ vết thực thi — SemLoc thay vào đó để LLM đề xuất bất biến.

2.3.2 Định vị lỗi bằng mô hình ngôn ngữ lớn

AutoFL [20] cho LLM gọi công cụ để duyệt mã và giải thích, ở cấp phương thức; SemLoc dùng AutoFL để chọn hàm trước khi xếp hạng dòng trên BugsInPy. AgentFL [21] mở rộng lên cấp dự án bằng nhiều tác tử. FlexFL [22] gần với ý tưởng của nhóm nhất: đưa kết quả SBFL làm gợi ý cho LLM mã nguồn mở, nhưng ở cấp phương thức trên mã Java. Điểm chung là bằng chứng thực thi được đưa vào dưới dạng văn bản; chưa nghiên cứu nào đo riêng phần độ chính xác đến từ bằng chứng và phần đến từ việc LLM tự đoán. Wu và cộng sự [8] cho thấy ChatGPT định vị lỗi tốt hơn hẳn khi được đưa nhật ký lỗi, tốt ở phạm vi một hàm nhưng giảm mạnh khi phải xét cả lớp. Ở bài toán sửa lỗi cấp kho mã, Agentless [23] cho thấy một quy trình hai bước đơn giản (định vị rồi sửa, không có tác tử tự chọn hành động) đạt kết quả tốt nhất trong các phương pháp mã nguồn mở trên SWE-bench Lite với chi phí thấp hơn nhiều — một tiền lệ cho câu hỏi "phức tạp có cần không".

2.3.3 Thực tế phát triển phần mềm

Các khảo sát lớn năm 2025 của Stack Overflow [5] và JetBrains [6] cho thấy phần lớn lập trình viên dùng hoặc định dùng trợ lý AI trong công việc, dù gần một nửa không tin độ chính xác của chúng. Bộ dữ liệu DevGPT [7] — các cuộc hội thoại thật giữa lập trình viên và ChatGPT được chia sẻ trên GitHub — cho thấy gỡ lỗi là một trong các nhóm việc thường được đem hỏi. Từ lâu, Parnin và Orso [24] đã chỉ ra rằng nhiều giả định của các kỹ thuật định vị lỗi tự động không khớp với cách lập trình viên thực sự gỡ lỗi, và khảo sát 386 người làm nghề của Kochhar và cộng sự [25] cho thấy người dùng có kỳ vọng cụ thể mà nghiên cứu chưa đáp ứng. Trong cách phát triển hướng kiểm thử [4], kiểm thử

được viết trước và mô tả ý định của mã — chính là thông tin mà một LLM cần để biết mã "sai" so với điều gì. Nhóm lấy cách làm phổ biến này — chạy kiểm thử, đưa kết quả thất bại cho LLM — làm mốc so sánh bắt buộc.

2.3.4 Bộ dữ liệu lỗi và tính hợp lệ của đánh giá

Defects4J [26] (Java) và BugsInPy [12] (Python) cung cấp lỗi thật kèm kiểm thử làm lộ lỗi; QuixBugs [27] gồm các chương trình nhỏ có lỗi một dòng. Pearson và cộng sự [28] cho thấy kết quả định vị lỗi trên lỗi nhân tạo không chuyển được sang lỗi thật, và cách chấm khi hòa điểm hay khi lỗi là thiếu mã có thể đảo thứ hạng giữa các kỹ thuật — đây là cơ sở cho phần kiểm tra cách đánh giá ở RQ1. Nhãn sai không phải chuyện hiếm: Northcutt và cộng sự [29] tìm thấy trung bình khoảng ba phần trăm nhãn sai trong tập kiểm tra của mười bộ dữ liệu học máy phổ biến, đủ để đảo thứ hạng giữa các mô hình. Agentless cũng phát hiện một phần đề bài của SWE-bench Lite có vấn đề và loại chúng ra khi đánh giá [23].

2.3.5 Bản sửa vượt kiểm thử nhưng không đúng

Trong sửa lỗi tự động, một bản vá làm mọi kiểm thử đạt được gọi là *hợp lệ bề ngoài* (plausible) — chưa chắc đã đúng. Qi và cộng sự [30] cho thấy phần lớn bản vá được báo cáo của các hệ thống sinh-và-kiểm-chứng (GenProg, RSRepair, AE) không đúng, nhiều bản chỉ tương đương với việc xóa bớt chức năng; Smith và cộng sự [31] gọi hiện tượng này là "quá khớp" bộ kiểm thử. Cùng một cơ chế chi phối mọi cách dùng kiểm thử làm trọng tài cho *vị trí* lỗi: bước vá thử của SemLoc và bộ kiểm chứng nhãn của nhóm.

2.3.6 Nhiễm dữ liệu và tái hiện khi dùng LLM

Sallou và cộng sự [32] liệt kê các mối đe dọa khi dùng LLM trong nghiên cứu kỹ nghệ phần mềm: mô hình đóng thay đổi theo thời gian, nhiễm dữ liệu, khó tái hiện. Ramos và

cộng sự [33] cho thấy LLM có dấu hiệu ghi nhớ các bộ dữ liệu lỗi phổ biến; khảo sát của Xu và cộng sự [34] đề xuất viết lại hoặc đổi tên để giảm nhiễm. Nhóm áp dụng trực tiếp: biến thể đổi tên định danh của SemFault-250 (RQ3), ghi nhật ký và lưu đệm mọi lượt gọi LLM để chạy lại được.

2.4 SO SÁNH VÀ KHOẢNG TRỐNG NGHIÊN CỨU

2.4.1 Bảng so sánh

Bảng 2.2 đặt ba bài được giao cạnh hướng tiếp cận của nhóm.

Bảng 2.2: So sánh ba bài báo được giao với hướng của nhóm

Tiêu chí	SemLoc [1]	EffiHolmes [2]	Báo cáo sự cố + LLM [3]	Nhóm M9
Loại lỗi	ngữ nghĩa	hiệu năng	sự cố (Java)	ngữ nghĩa
Tín hiệu	ràng buộc do LLM sinh + chạy kiểm thử	phân tích hiệu năng ở hai mức tải	vết ngăn xếp + xếp hạng	kiểm thử thất bại đưa thẳng cho LLM; tách đóng góp của từng tín hiệu
Lỗi thật	7 lỗi có kết quả cấp dòng	140 vấn đề	38 lỗi công nghiệp	31 lỗi BugsInPy, 3 lần chạy
Kiểm tra bộ dữ liệu	không	—	—	kiểm chứng bằng thực thi và rà soát toàn bộ nhãn
Lặp và kiểm định	1 lần, không kiểm định	—	—	3 lần, McNemar, bootstrap theo cụm

2.4.2 Khoảng trống và hướng của nhóm

Ba khoảng trống định hướng toàn bộ tiểu luận. Thứ nhất, các hệ thống định vị lỗi bằng LLM tuyên bố "neo vào thực thi" nhưng chưa ai đo phần độ chính xác thật sự đến từ bằng chứng thực thi. Thứ hai, kết quả được báo cáo từ một lần chạy với cách chấm lạc quan trên một bộ dữ liệu có nhãn chưa được kiểm tra. Thứ ba, chưa có so sánh với cách làm đơn giản mà lập trình viên vẫn dùng: một prompt duy nhất có kiểm thử thất bại. Từ đó, nhóm học và áp dụng: từ SemLoc — giả thuyết do LLM sinh nhưng kiểm thử phán xử, và dùng chính mã nguồn gốc để tái hiện; từ EffiHolmes — đối chiếu hai lần thực thi để lọc nhiễu (ý tưởng của CovGate) và khuôn câu hỏi nghiên cứu; từ báo cáo công nghiệp — đưa thông tin có sẵn vào prompt thay vì xây quy trình phức tạp; từ Pearson và cộng sự, Northcutt và cộng sự — kiểm tra cách chấm và kiểm tra chính bộ nhãn; từ Qi và cộng sự — không tin một bản sửa chỉ vì nó làm kiểm thử đạt.

Chương 3. Cơ sở lý thuyết

3.1 ĐỊNH VỊ LỖI DỰA TRÊN PHỔ ĐỘ PHỦ

3.1.1 Ma trận độ phủ và các công thức nghi ngờ

Cho chương trình P và bộ kiểm thử $T = T_f \cup T_p$ gồm các kiểm thử thất bại T_f và thành công T_p. Với mỗi dòng s, ký hiệu $e_f(s)$, $e_p(s)$ là số kiểm thử thất bại, thành công chạy qua s; $n_f(s)$, $n_p(s)$ là số kiểm thử thất bại, thành công không chạy qua s. Ma trận độ phủ ghi lại, với mỗi cặp (kiểm thử, dòng), kiểm thử đó có chạy qua dòng đó hay không. SBFL gán cho mỗi dòng một điểm nghi ngờ; ba công thức dùng trong tiểu luận là Tarantula [9], Ochiai [10] và DStar [13]:

$$\text{Tarantula}(s) = \frac{e_f/(e_f + n_f)}{e_f/(e_f + n_f) + e_p/(e_p + n_p)}, \tag{3.1}$$

$$\text{Ochiai}(s) = \frac{e_f}{\sqrt{(e_f + n_f)\,(e_f + e_p)}}, \tag{3.2}$$

$$\text{DStar}(s) = \frac{e_f^*}{e_p + n_f}, \quad * = 2. \tag{3.3}$$

Các dòng được xếp giảm dần theo điểm; dòng không được kiểm thử thất bại nào chạy qua ($e_f = 0$) có điểm 0 và trong tiểu luận không được xếp hạng.

3.1.2 Chỉ số đánh giá và cách xử lý hòa điểm

Với mỗi chương trình, *hạng* của lỗi là hạng tốt nhất của một dòng lỗi chuẩn trong danh sách xếp hạng. Acc@k là tỉ lệ chương trình có hạng không vượt quá k:

$$\text{Acc@}k = \frac{1}{|\mathcal{P}|} \sum_{p \in \mathcal{P}} \mathbb{1}[\,\text{rank}_p \le k\,], \tag{3.4}$$

trong đó $\mathcal{P}$ là tập chương trình *cố định*: chương trình mà công cụ không chạy được vẫn nằm trong mẫu số và được tính là trượt. Khi nhiều dòng cùng điểm, có ba cách phá hòa: lấy vị trí đầu (lạc quan, theo thứ tự dòng), vị trí cuối (bi quan) hoặc *hạng trung bình* của nhóm hòa. Tiểu luận dùng hạng trung bình, theo khuyến nghị của các nghiên cứu về đánh giá định vị lỗi [28, 14]; cách chấm của tác giả SemLoc dùng vị trí đầu. Chỉ số chính chấm *khớp đúng dòng*: dòng dự đoán phải nằm trong tập dòng lỗi chuẩn. Từ lần báo cáo 2, tiểu luận báo cáo thêm một chỉ số phụ, *Acc@1 nới lỏng*, chấp nhận cả dòng kề cạnh khi hai dòng là đầu một câu lệnh ghép và câu lệnh nó bảo vệ (Mục 5.2.2).

3.2 LỖI NGỮ NGHĨA

3.2.1 Vì sao độ phủ mất tín hiệu

Lỗi ngữ nghĩa là lỗi làm chương trình tính sai giá trị trong khi vẫn chạy qua cùng các câu lệnh như khi đúng — ví dụ `updated = cap - 1` thay vì `updated = cap`. Khi cả kiểm thử thành công và thất bại đều chạy qua dòng lỗi, $e_p(s) > 0$ làm điểm Ochiai của dòng lỗi giảm và ngang với các dòng xung quanh; SBFL khi đó gần như đoán mò trong phạm vi hàm.

3.2.2 Định nghĩa khách quan bằng độ phủ

Để phân tích kết quả theo loại lỗi, tiểu luận dùng một định nghĩa tự động, chỉ dựa trên độ phủ của các dòng lỗi chuẩn:

- **lỗi ngữ nghĩa** (semantic): có dòng lỗi được ít nhất một kiểm thử thất bại *và* một kiểm thử thành công chạy qua — độ phủ không phân biệt được;
- **lộ qua độ phủ** (coverage-revealed): kiểm thử thất bại chạy tới dòng lỗi nhưng không kiểm thử thành công nào chạy tới;
- **không thực thi** (not executed): không kiểm thử thất bại nào chạy tới dòng lỗi

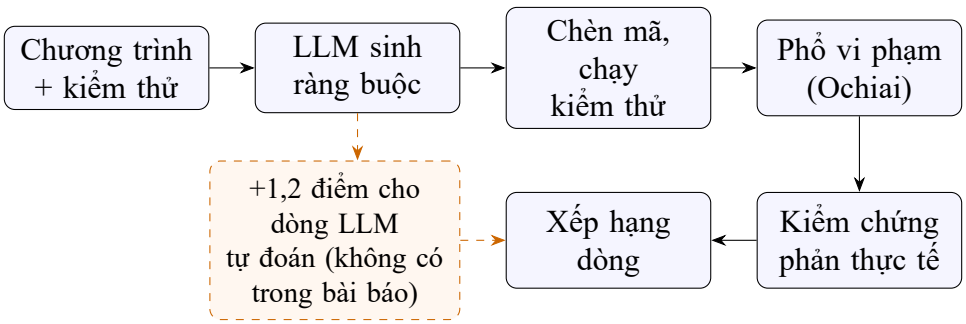
chuẩn — thường do dòng lỗi chuẩn bị đặt sai hoặc là dòng trống.

Nhãn được tính lại theo đúng bộ dòng lỗi chuẩn đang dùng để chấm (gốc hoặc đã kiểm chứng).

3.3 SEMLOC: PHỔ VI PHẠM RÀNG BUỘC NGỮ NGHĨA

3.3.1 Quy trình bốn bước

Hình 3.1 tóm tắt quy trình của SemLoc [1]. Mỗi ràng buộc c do LLM sinh ra là một bộ ⟨mã, loại, vùng, điểm neo, biểu thức, ý đồ⟩; biểu thức được chèn vào chương trình dưới dạng một lời gọi kiểm tra tại điểm neo. Chạy bộ kiểm thử cho ma trận "ràng buộc × kiểm thử" ghi ràng buộc nào bị vi phạm trong kiểm thử nào; thay "dòng" bằng "ràng buộc" trong công thức (3.2) cho điểm của từng ràng buộc, và điểm của một dòng là điểm lớn nhất trong các ràng buộc được quy về dòng đó. Bước kiểm chứng phản thực tế loại ràng buộc bị vi phạm cả trong kiểm thử thành công, rồi cho LLM vá thử tại dòng được quy cho từng ràng buộc còn lại và chạy lại kiểm thử.



Hình 3.1: Quy trình của SemLoc (theo [1]); khung nét đứt là thành phần nhóm phát hiện trong mã nguồn

3.3.2 Điểm thưởng cho dòng LLM tự đoán

Khi đọc mã nguồn công khai, nhóm thấy một thành phần không được mô tả trong bài báo: ngoài các ràng buộc, LLM còn trả về một dòng lỗi mà nó tự đoán (trường `fault_line`),

và dòng này được cộng thêm 1,2 điểm — lớn hơn điểm tối đa 1 của một ràng buộc. Vì vậy, dòng LLM tự đoán *luôn* đứng hạng 1, bất kể phổ vi phạm ra sao. Thành phần này là trọng tâm của câu hỏi "grounded hay guessed" ở các chương sau.

3.4 PHÂN TÍCH TĨNH VÀ PHÂN TÍCH ĐỘNG

Phân tích tĩnh xét mã nguồn mà không chạy chương trình. Trình kiểm tra phong cách mã (lint) như ruff phát hiện mẫu mã dễ lỗi hoặc khó đọc (biến không dùng, so sánh thừa, cách viết dài dòng); trình kiểm tra kiểu như pyrefly suy kiểu và báo chỗ kiểu không khớp với chú thích. Cả hai chỉ biết mã *được viết* thế nào, không biết mã *được mong đợi* làm gì: viết > thay cho >= hay trừ phí hai lần đều hợp lệ về cú pháp và về kiểu. Phân tích động lấy thông tin từ việc chạy chương trình: kết quả kiểm thử, độ phủ, giá trị biến. Với lỗi ngữ nghĩa, chỉ kiểm thử — bản đặc tả do lập trình viên viết — mới chứa thông tin về ý định, nên định vị lỗi ngữ nghĩa thuộc về phân tích động; phân tích tĩnh sẽ được dùng như một mốc so sánh ở giai đoạn sau.

3.5 CÔNG CỤ THỐNG KÊ

3.5.1 Kiểm định cặp

Mọi so sánh giữa hai phương pháp đều là so sánh *cặp* trên cùng các chương trình. Với chỉ số nhị phân (trúng hay trượt top-k), kiểm định McNemar dùng số chương trình chỉ phương pháp A trúng (b) và chỉ phương pháp B trúng (c), bản chính xác khi $b + c$ nhỏ. Với hạng nghịch đảo ($1/\text{rank}$, trượt là 0), kiểm định dấu hạng Wilcoxon cùng hệ số tương quan hạng hai chuỗi ghép cặp làm độ lớn hiệu ứng:

$$r_{rb} = \frac{R^+ - R^-}{R^+ + R^-}, \tag{3.5}$$

với R^+, R^- là tổng hạng của các chênh lệch dương, âm (bỏ chênh lệch bằng 0).

3.5.2 Khoảng tin cậy bootstrap theo cụm và giá trị Shapley

Mỗi cấu hình được chạy ba lần. Kết quả của một chương trình được lấy trung bình qua ba lần *trước khi* lấy mẫu lại, nên đơn vị lấy mẫu là chương trình chứ không phải lần chạy. Vì nhiều chương trình của SemFault-250 sinh ra từ cùng một khuôn mẫu, khoảng tin cậy 95% được tính bằng bootstrap *theo cụm khuôn mẫu* (lấy lại cả cụm), với 10000 lần lấy mẫu và hạt giống cố định.

Để đo phần đóng góp của từng tín hiệu, tiểu luận dùng giá trị Shapley. Với tập tín hiệu N và hàm giá trị $v(S)$ là Acc@k khi chỉ bật các tín hiệu trong S,

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!\,(|N| - |S| - 1)!}{|N|!} \Big(v(S \cup \{i\}) - v(S) \Big), \tag{3.6}$$

và $\sum_i \phi_i = v(N)$. Vì Acc@k là trung bình các giá trị trúng/trượt theo chương trình, giá trị Shapley tổng bằng trung bình giá trị Shapley của từng chương trình; tính chất này cho phép tính khoảng tin cậy bootstrap cho từng ϕ_i.

Chương 4. Phương pháp và thiết kế thực nghiệm

4.1 CÂU HỎI NGHIÊN CỨU VÀ GIẢ THUYẾT

4.1.1 Ba câu hỏi nghiên cứu

RQ1 (tái hiện và kiểm tra cách đánh giá): các con số công bố của SemLoc có tái hiện được từ dữ liệu và mã nguồn của tác giả không; chúng thay đổi thế nào khi (a) chấm điểm công bằng, (b) bỏ các thành phần không được mô tả trong bài báo, (c) kiểm tra lại chính các dòng lỗi chuẩn của bộ dữ liệu, (d) chạy lại với mô hình khác?

RQ2 (LLM thật sự cần gì?): trên cùng các chương trình và cùng mô hình, (a) một prompt duy nhất có kiểm thử thất bại so với quy trình nhiều bước SemLoc thế nào về độ chính xác và chi phí; (b) trong prompt đó, thông tin nào tạo ra khác biệt — kiểm thử thất bại hay độ phủ từng dòng; (c) các cách bổ sung (CovGate cho SemLoc; biểu quyết nhiều câu trả lời) và công cụ phân tích tĩnh có thêm được gì; (d) bao nhiêu phần độ chính xác của SemLoc đến từ bằng chứng thực thi, bao nhiêu từ dòng LLM tự đoán?

RQ3 (độ bền): các kết luận của RQ2 có còn đúng (a) trên lỗi thật của BugsInPy, (b) khi đổi tên mọi định danh và xóa chú thích của chương trình (kiểm tra ghi nhớ), (c) trên năm mô hình thuộc các họ khác nhau, (d) trên các bộ nhãn khác nhau?

4.1.2 Giả thuyết và tiêu chí bác bỏ

H1: *ở hạng 1, độ chính xác của SemLoc đến từ dòng LLM tự đoán; bằng chứng thực thi đóng góp ở top-3*. Bác bỏ nếu riêng phổ vi phạm đạt Acc@1 không thấp hơn SemLoc, hoặc nếu ở top-3 phổ không làm tăng độ chính xác so với riêng dòng đoán. **H2:** *một prompt có kiểm thử thất bại không kém quy trình nhiều bước*; bác bỏ nếu khoảng tin cậy của chênh lệch Acc@1 hoặc Acc@3 so với SemLoc nằm hẳn dưới 0. **H3:** *thêm độ phủ từng dòng vào prompt cải thiện kết quả*; bác bỏ nếu khoảng tin cậy của chênh lệch không

nằm hẳn trên 0. Mọi giả thuyết được kiểm trên cả ba bộ nhãn.

4.2 DỮ LIỆU

4.2.1 SemFault-250 và cách chia tập

SemFault-250 [1] gồm 250 chương trình Python một hàm, mỗi chương trình có đúng một lỗi được cài vào và khoảng mười kiểm thử. Nhóm chia cố định 50 chương trình làm tập phát triển (dev) — chỉ dùng để chọn tham số — và 200 chương trình làm tập kiểm tra (test) để báo cáo, phân tầng theo nhãn lỗi (mục 3.2) với hạt giống cố định. Vì nhiều chương trình được sinh từ cùng một khuôn mẫu, nhóm gom các chương trình có cấu trúc giống nhau thành cụm (độ tương đồng 4-gram Jaccard từ bốn phần năm trở lên, trên dạng chuẩn hóa của mã) để dùng trong bootstrap theo cụm.

4.2.2 SemFault-250 đã sửa: kiểm chứng dòng lỗi chuẩn bằng thực thi

Kiểm tra dòng lỗi chuẩn cho thấy 56 dòng là dòng trống và 65 chương trình không có kiểm thử thất bại nào chạy tới dòng lỗi chuẩn. Nhóm xây một *bộ phán định sửa tối thiểu*: một dòng được coi là vị trí lỗi nếu thay nó bằng một dòng khác (hoặc chèn một dòng vào dòng trống, với lỗi thiếu mã) với *thay đổi nhỏ* làm *mọi* kiểm thử đạt. LLM chỉ đề xuất các bản sửa ứng viên (GPT-6 luna, hai rồi bốn lần thử ở $T = 0,8$; lượt thứ hai với Claude Sonnet 5.5 cho các dòng còn chưa kiểm chứng); kiểm thử quyết định. Bản sửa được coi là nhỏ khi khoảng cách chỉnh sửa theo từ tố (định danh, số, từng ký tự dấu) so với dòng gốc tối đa là hai hoặc tối đa một phần ba số từ tố của dòng. Các dòng lân cận (trong phạm vi bốn dòng) chỉ được tìm khi không kiểm thử thất bại nào chạy tới dòng chuẩn gốc; vì vậy bộ phán định không phát hiện được trường hợp hai dòng cùng sửa được lỗi. Kết quả: 82 chương trình được *xác nhận* (sửa được ngay tại dòng chuẩn), 46 được *sửa vị trí* (dòng chuẩn không chạy tới được nhưng một dòng gần đó sửa được), 111 *chưa kiểm chứng được* (dòng chuẩn chạy tới được nhưng không bản sửa đề xuất nào đạt)

và 11 *không kiểm chứng được* (bị loại). Vị trí "sửa" nằm trên dòng trống không phải câu lệnh nào để xếp hạng nên bị loại; bộ dòng lỗi chuẩn đã sửa còn 226 chương trình. Mọi kết quả được báo cáo trên cả bộ gốc và bộ đã sửa.

4.2.3 Lỗi thật BugsInPy

Nhóm dựng môi trường cho các lỗi của dự án youtube-dl trong BugsInPy [12] và giữ những lỗi thỏa đủ bốn điều kiện: môi trường cài được, kiểm thử của lỗi tái hiện được thất bại trên phiên bản lỗi, đạt trên phiên bản đã sửa, và đo được độ phủ — được 31 lỗi. Hàm đích là hàm chứa dòng bị lập trình viên sửa mà kiểm thử thất bại được khai báo thực sự chạy qua; dòng lỗi chuẩn là các dòng bị sửa trong hàm đó. Kiểm thử thất bại do môi trường (không được BugsInPy khai báo) bị loại khỏi ma trận độ phủ.

4.2.4 Biến thể đổi tên định danh

Để kiểm tra khả năng mô hình ghi nhớ SemFault-250 [33, 34], nhóm đổi tên mọi định danh do chương trình tự định nghĩa (hàm, tham số, biến, hằng) thành tên vô nghĩa, xóa chú thích và docstring nhưng *giữ nguyên số dòng*, chỉ giữ các tên mà kiểm thử phụ thuộc. Mỗi biến thể được kiểm tra tự động: cùng số dòng, cùng kết quả đạt/không đạt của từng kiểm thử như bản gốc; chương trình nào lệch bị loại. Kết quả là 199 chương trình của tập test.

4.3 CÁC PHƯƠNG PHÁP SO SÁNH

4.3.1 Các phương pháp có sẵn

- **SBFL Ochiai, DStar** trên ma trận độ phủ do nhóm thu thập (mỗi kiểm thử một ngữ cảnh độ phủ).

- **LLM hỏi thẳng**: chế độ dự đoán trực tiếp của SemLoc — một lượt gọi chỉ có mã nguồn chương trình, LLM trả về các dòng nghi ngờ; không có thông tin kiểm thử nào.
- **SemLoc** như phát hành, chạy qua mã nguồn gốc của tác giả, chỉ thay lớp gọi LLM bằng cổng của nhóm (cùng prompt, cùng logic).

4.3.2 LLM + kiểm thử thất bại: phương pháp đề xuất

Phương pháp chính của nhóm làm điều mà các khảo sát cho thấy nhiều lập trình viên đang làm: một lượt gọi LLM duy nhất (nhiệt độ $T = 0,8$) với mã nguồn đánh số dòng, số kiểm thử đạt/không đạt, câu "chương trình có đúng một dòng lỗi", và tên cùng dòng thông báo lỗi đầu tiên của mỗi kiểm thử thất bại; LLM trả về năm dòng nghi ngờ nhất dưới dạng JSON (nội dung prompt có trong kho mã). Prompt được thiết kế trước khi xem kết quả trên tập test và không được tinh chỉnh. Ý tưởng đưa thông tin sẵn có vào prompt thay vì xây quy trình nhiều bước lấy từ báo cáo công nghiệp [3] và các nghiên cứu cho thấy nhật ký lỗi giúp LLM định vị tốt hơn [8].

4.4 KIỂM TRA CÁCH ĐÁNH GIÁ VÀ PHÂN RÃ ĐÓNG GÓP

4.4.1 Kiểm tra cách đánh giá của SemLoc

Từ nhật ký công bố của tác giả, nhóm tính lại mọi cấu hình theo hai cách: *cách chấm của tác giả* (phá hòa theo thứ tự dòng, gỡ lỗi chia đôi trúng bất kỳ dòng nào tính là hạng 1, chương trình chạy lỗi bị loại khỏi mẫu số) và *cách chấm công bằng* (hạng trung bình khi hòa, mẫu số cố định, công thức (3.4)). Nhóm cũng loại bỏ lần lượt các thành phần không được mô tả trong bài báo (điểm thưởng dòng đoán) và kiểm tra bước kiểm chứng phản thực tế có làm thay đổi thứ hạng hay không.

4.4.2 Phân rã "grounded hay guessed"

Điểm dòng của SemLoc là tổng của hai tín hiệu: phổ vi phạm ràng buộc (bằng chứng thực thi) và điểm thưởng cho dòng LLM tự đoán. Nhóm định nghĩa trò chơi hai người chơi {dòng đoán, phổ}: $v(S)$ là Acc@k khi chỉ giữ các tín hiệu trong S; khi giữ cả hai, điểm dòng trùng đúng với SemLoc như phát hành. Giá trị Shapley (3.6) chia Acc@k của SemLoc cho hai tín hiệu; một trò chơi mở rộng bốn người chơi thêm độ phủ (Ochiai) và xếp lại bằng kiểm chứng phản thực tế.

4.5 THIẾT LẬP THỰC NGHIỆM

4.5.1 Mô hình và hạ tầng

Mô hình chính là Gemini 3.8 Flash; RQ3 thêm GPT-6 luna, Claude Sonnet 5.5, gpt-oss-120b (mã nguồn mở) và Gemini 3.5 Flash-Lite trên một tập con cố định 50 chương trình của tập test; Claude Sonnet 4.6 chỉ dùng để đối chiếu với nhật ký của tác giả. Mọi mô hình chạy ở mức suy luận mặc định của nhà cung cấp, qua một cổng tương thích OpenAI; SemLoc chạy ở nhiệt độ $T = 0,8$ như cấu hình chính của bài báo. Mọi lượt gọi LLM được ghi nhật ký (mô hình thực sự trả lời, số token, thời gian, lỗi) và lưu đệm theo nội dung và mã lượt chạy, nên toàn bộ thực nghiệm phát lại được mà không gọi mạng.

4.5.2 Lặp, kiểm tra sức khỏe lượt chạy và thống kê

Mỗi cấu hình có LLM chạy 3 lần độc lập; mỗi lần là một mã lượt chạy và một thư mục làm việc riêng. Một lượt chạy chỉ được dùng khi đã chạy xong và qua kiểm tra sức khỏe: không còn lỗi gọi LLM chưa khắc phục, không có chương trình nào lỗi ở bước kiểm chứng phản thực tế. Kết quả báo cáo là trung bình $\pm$ độ lệch chuẩn qua 3 lần; so sánh cặp dùng chênh lệch Δ kèm khoảng tin cậy bootstrap theo cụm, McNemar cho từng lần chạy và Wilcoxon kèm r_{rb} (mục 3.5). Chi phí được đo bằng số lượt gọi, số token và thời

gian trên mỗi chương trình.

Chương 5. Kết quả thực nghiệm

Mọi con số trong chương là phần trăm (Acc@k) hoặc điểm phần trăm (Δ), sinh tự động từ dữ liệu; khoảng trong ngoặc vuông là khoảng tin cậy 95% bootstrap theo cụm khuôn mẫu. Có khoảng ba mươi phép so sánh như vậy và nhóm không hiệu chỉnh đa so sánh, nên các khoảng sát 0 cần được đọc thận trọng. Trừ khi ghi khác, kết quả là của Gemini 3.8 Flash, ba lần chạy, trên tập test.

5.1 RQ1 – TÁI HIỆN VÀ KIỂM TRA CÁCH ĐÁNH GIÁ

5.1.1 Tái hiện và hai cách chấm điểm

Tính lại từ nhật ký công bố khớp từng con số của cả 13 cấu hình trong bảng kết quả của SemLoc. *Chạy lại* SemLoc với chính mô hình của bài báo (Claude Sonnet 4.6) trên tập dev thì không khớp: Acc@1 32,0% và Acc@3 44,0% so với 36,0% và 68,0% trong nhật ký trên cùng các chương trình; ràng buộc sinh ra ít bị vi phạm trong kiểm thử thất bại hơn, và nhóm không tìm thấy nguyên nhân ở hạ tầng của mình.

Bảng 5.1 so sánh cách chấm của tác giả với cách chấm công bằng. Ba điểm nổi bật:

- **Gỡ lỗi chia đôi**: mã chấm điểm công bố cho 38,0% (mọi lần trúng tính là hạng 1), trong khi bài báo ghi 0 và cách chấm công bằng cho 0,0%, vì phương pháp này không xếp hạng dòng nào.
- **SBFL bị đánh giá thấp ở top-5**: Acc@5 của Ochiai là 30,4% theo cách của tác giả và 48,0% khi dùng hạng trung bình.
- **SemLoc** gần như không đổi khi chấm công bằng, nhưng **bỏ điểm thưởng cho dòng LLM tự đoán** — thành phần không có trong bài báo — làm Acc@1 đi từ 42,8% xuống 30,0% (Claude) và từ 39,6% xuống 27,2% (Gemini).

Bảng 5.1: Kết quả công bố tính lại theo hai cách chấm (nhật ký của tác giả, 250 chương trình)

	Cách chấm của tác giả			Cách chấm công bằng		
Cấu hình (cả 250 chương trình)	Acc@1	Acc@3	Acc@5	Acc@1	Acc@3	Acc@5
SemLoc, Claude Sonnet 4,6, $T{=}0,8$	42,8	68,0	76,8	42,8	65,6	76,8
SemLoc, Gemini 2,5 Pro, $T{=}0,8$	39,8	64,7	73,5	39,6	64,4	72,8
SemLoc Claude $T{=}0,8$ không có điểm thưởng dòng đoán	–	–	–	30,0	55,6	66,4
SemLoc Gemini $T{=}0,8$ không có điểm thưởng dòng đoán	–	–	–	27,2	55,6	64,0
SemLoc không CF (lượt chạy riêng), Claude	30,8	59,6	68,8	–	–	–
LLM hỏi thẳng, Claude	35,2	41,2	41,2	35,2	41,2	41,2
SBFL Ochiai	6,4	13,2	30,4	4,0	16,0	48,0
SBFL DStar (tính lại)	–	–	–	4,0	16,0	48,0
Gỡ lỗi chia đôi (DD)	38,0	38,0	38,0	0,0	0,0	0,0

5.1.2 Kiểm chứng phản thực tế và lỗi của gói mã công bố

Trong mã nguồn công bố, kết quả của bước kiểm chứng phản thực tế được tính ra nhưng *không dùng để xếp hạng*: thứ hạng có và không có bước này trùng nhau ở cả 250 chương trình của cả năm cấu hình có dữ liệu, dù bước này chiếm 42% số lượt gọi LLM của SemLoc. Con số "không có kiểm chứng phản thực tế" trong bài báo đến từ một lượt chạy riêng không công bố điểm từng chương trình; Acc@1 của nó (30,8%) gần bằng SemLoc khi bỏ điểm thưởng, gợi ý một khác biệt về cấu hình hơn là tác dụng của bước này, nhưng nhóm không xác định được. Bộ chạy BugsInPy của SemLoc có thêm hai lỗi: (i) chỉ số "hạng tốt nhất" nó báo cáo là hạng của *chính dòng LLM đoán*, không phải của dòng lập trình viên đã sửa; (ii) bước vá thử sửa tệp nguồn rồi khôi phục trong cùng một giây với cùng kích thước tệp, để lại bản biên dịch Python (`.pyc`) của phiên bản *đã vá*, khiến lỗi "biến mất" ở lượt chạy sau. Nhóm đã sửa lỗi thứ hai trong bộ chạy của mình và chấm lại BugsInPy theo dòng đã sửa thật.

5.1.3 Ảnh hưởng của dòng lỗi chuẩn

56 nhãn gốc nằm trên dòng trống và 65 chương trình không có kiểm thử thất bại nào chạy tới dòng lỗi chuẩn. Trên lượt chạy của nhóm, chuyển từ nhãn gốc (200 chương trình) sang bộ đã sửa bằng thực thi làm Acc@1 của SemLoc đi từ 36,8% lên 52,3%; một phần mức tăng đến từ mẫu số nhỏ hơn, vì bộ đã sửa bỏ các lỗi không kiểm chứng được. Việc rà soát toàn bộ nhãn đang được thực hiện.

5.2 RQ2 – LLM THẬT SỰ CẦN GÌ?

5.2.1 Một prompt có kiểm thử thất bại so với quy trình nhiều bước

Bảng 5.2 là kết quả chính trên bộ nhãn gốc và bộ đã sửa. **LLM + kiểm thử thất bại** — phương pháp đề xuất — thắng SemLoc ở Acc@1 trên cả hai bộ: +7,7 [+3,5; +12,4] trên bộ gốc và +10,2 [+5,3; +16,0] trên bộ đã sửa. Ở Acc@3, nó hơn +14,5 [+8,5; +20,8] điểm trên bộ đã sửa; trên toàn bộ nhãn gốc, mức hơn +7,2 có khoảng tin cậy chạm 0, vì nhãn gốc chứa các lỗi mà không kiểm thử nào chạy tới. Phương pháp đề xuất tốn 2,0 nghìn token mỗi chương trình, so với 14,0 nghìn của SemLoc. SBFL thấp nhất trên cả hai bộ nhãn.

Bảng 5.2: Kết quả chính trên bộ nhãn gốc và bộ đã sửa (tập test, Gemini 3.8 Flash, trung bình 3 lần chạy) và chi phí token mỗi chương trình

	Bản gốc ($n = 200$)		Đã sửa ($n = 179$)		
Phương pháp	Acc@1	Acc@3	Acc@1	Acc@3	nghìn token
SBFL Ochiai	3,0	15,5	3,9	19,0	0
LLM hỏi thẳng (chỉ có mã)	40,0	43,0	56,8	62,8	2,5
SemLoc (nhiều bước)	36,8	60,3	52,3	75,4	14,0
LLM + kiểm thử thất bại (nhóm đề xuất)	44,5	67,5	62,6	89,9	2,0

5.2.2 Độ nhạy với quy ước gán nhãn

Acc@k ở trên chấm *khớp đúng dòng*: chỉ vào dòng ngay cạnh dòng lỗi chuẩn cũng là trượt. Khi đọc các ca trượt, nhóm thấy phần lớn là chỉ vào dòng kề cạnh, thường là cặp *dòng điều kiện – câu lệnh nó bảo vệ*: lỗi cài vào nằm ở câu lệnh bên trong `if` còn nhãn ghi dòng `if`, hoặc ngược lại. Vì vậy nhóm báo cáo thêm một chỉ số phụ, *Acc@1 nới lỏng*: dòng hạng một cũng được tính đúng khi nó và một dòng lỗi chuẩn là đầu một câu lệnh ghép (`if`, `for`, `while`, `with`, `try`) và dòng đầu tiên của thân câu lệnh đó; *cận trên* chấp nhận thêm phần còn lại của câu lệnh đầu tiên trong thân nếu nó dài không quá ba dòng. Thứ hạng, các lần chạy và tham số giữ nguyên, chỉ tập dòng được chấp nhận thay đổi. Chỉ số phụ được đặt ra sau khi đã thấy kết quả, nên nhóm chỉ dùng nó để kiểm tra độ bền của kết luận; mọi giả thuyết vẫn được kiểm định trên chỉ số chính.

Bảng 5.3: Acc@1 khớp đúng dòng, nới lỏng (dòng điều kiện – dòng đầu của thân) và cận trên (tập test, Gemini 3.8 Flash, trung bình 3 lần chạy)

	Bản gốc ($n = 200$)			Đã sửa ($n = 179$)		
Phương pháp	Khớp đúng	Nới lỏng	Cận trên	Khớp đúng	Nới lỏng	Cận trên
SBFL Ochiai	3,0	10,5	12,5	3,9	14,5	16,8
LLM hỏi thẳng (chỉ có mã)	40,0	55,5	57,5	56,8	71,9	74,1
SemLoc (nhiều bước)	36,8	56,2	58,2	52,3	73,9	76,2
LLM + kiểm thử thất bại (nhóm đề xuất)	44,5	62,2	64,2	62,6	80,1	82,3
+ độ phủ từng dòng	40,2	61,5	63,5	57,2	79,3	81,6

Bảng 5.3 cho thấy nới lỏng nâng Acc@1 của mọi phương pháp dùng LLM lên rõ rệt: trên bộ đã sửa, SemLoc từ 52,3% lên 73,9%, LLM + kiểm thử thất bại từ 62,6% lên 80,1%. Phần lớn các lần "trượt" vì vậy là trượt theo quy ước gán nhãn chứ không phải chỉ sai chỗ. Kết luận chính không đổi: LLM + kiểm thử thất bại vẫn hơn SemLoc +6,2 [+2,4; +10,5] điểm Acc@1 trên bộ đã sửa và +6,0 [+2,4; +10,1] trên bộ gốc. Khoảng cách hẹp lại so với khi chấm khớp đúng (+10,2 điểm trên bộ đã sửa): một phần lợi thế của phương pháp đề xuất là chọn đúng dòng mà nhãn ghi trong cặp dòng điều kiện – thân.

Riêng LLM hỏi thẳng, vốn hơn SemLoc khi chấm khớp đúng, trở nên ngang SemLoc khi nới lỏng (−2,1 [−5,8; +1,4] điểm trên bộ đã sửa).

5.2.3 Grounded hay guessed: giá trị biên và phân rã

Vì điểm thưởng lớn hơn mọi điểm ràng buộc, thêm phổ vào dòng đoán *không thể* thay đổi dòng đầu tiên: riêng dòng đoán và SemLoc cùng đạt Acc@1 52,3% trên bộ đã sửa, còn riêng phổ đạt 43,2%. Giá trị Shapley của phổ ở Acc@1 (Bảng 5.4, 21,6 điểm) vì vậy là phần công cho điều phổ *sẽ* đạt được nếu không có dòng đoán, không phải đóng góp vào dòng đầu của SemLoc. Ở top-3, phổ có tác dụng thật: dòng đoán chỉ nêu một dòng nên dừng ở 52,3%, riêng phổ đạt 70,8%, và cả hai cùng đạt 75,4%. Kết luận: **hạng 1 là "guessed" theo thiết kế; bằng chứng thực thi của SemLoc có tác dụng ở top-3** — nhưng một prompt có kiểm thử thất bại đạt top-3 cao hơn mà không cần phổ. Trên nhật ký của tác giả (Claude), phân rã cho 42,8% = 27,8 (dòng đoán) + 15,0 (phổ).

Bảng 5.4: Phân rã Shapley độ chính xác của SemLoc (lượt chạy của nhóm, 3 lần)

Dòng lỗi chuẩn	Chỉ số	Tổng	Dòng LLM đoán	Phổ ngữ nghĩa	Độ phủ	CF
SemLoc như phát hành (dòng đoán + phổ)						
Bản gốc ($n = 200$)	Acc@1	36,8	22,2 [16,5; 28,4]	14,6 [11,8; 17,5]	–	–
Bản gốc ($n = 200$)	Acc@3	60,3	19,8 [16,1; 23,8]	40,6 [35,6; 45,3]	–	–
Đã sửa ($n = 179$)	Acc@1	52,3	30,7 [23,5; 37,8]	21,6 [17,4; 26,1]	–	–
Đã sửa ($n = 179$)	Acc@3	75,4	28,5 [23,4; 33,5]	46,9 [41,9; 51,7]	–	–
Mọi tín hiệu (+ Ochiai thô, + xếp lại bằng CF)						
Bản gốc ($n = 200$)	Acc@1	35,0	22,7 [17,4; 28,3]	12,5 [10,0; 15,2]	-0,5 [-1,5; 0,6]	0,3 [-0,6; 1,1]
Bản gốc ($n = 200$)	Acc@3	59,0	17,6 [14,2; 21,4]	35,8 [31,6; 40,2]	4,6 [0,1; 9,0]	1,1 [0,5; 1,8]
Đã sửa ($n = 179$)	Acc@1	49,9	31,9 [25,3; 38,5]	18,6 [14,4; 22,9]	-1,1 [-2,3; 0,2]	0,5 [-0,7; 1,5]
Đã sửa ($n = 179$)	Acc@3	80,6	25,9 [20,8; 31,0]	44,4 [39,7; 49,3]	9,4 [5,8; 13,4]	0,9 [0,3; 1,7]

5.3 RQ3 – ĐỘ BỀN

RQ3 đang được thực hiện: lỗi thật của BugsInPy (dựng môi trường cho từng lỗi), biến thể đổi tên định danh của SemFault-250 và các họ mô hình khác; kết quả sẽ được trình bày ở lần báo cáo sau.

5.4 TÓM TẮT TRẢ LỜI CÁC CÂU HỎI NGHIÊN CỨU

Bảng 5.5: Trả lời các câu hỏi nghiên cứu đến thời điểm báo cáo

Câu hỏi	Trả lời
RQ1	Tính lại từ nhật ký khớp toàn bộ, chạy lại thì không; điểm thưởng ẩn quyết định dòng đầu; bước kiểm chứng phản thực tế không đổi thứ hạng; mã chấm điểm lạc quan với gỡ lỗi chia đôi và bi quan với SBFL; nhãn gốc có nhiều dòng trống và lỗi không được chạy tới.
RQ2	Một prompt có kiểm thử thất bại thắng quy trình nhiều bước ở Acc@1 trên bộ gốc và bộ đã sửa, rẻ hơn khoảng bảy lần. Hạng 1 của SemLoc là “guessed” theo thiết kế. Đang thực hiện: thông tin nào trong prompt tạo ra khác biệt, các cách bổ sung.
RQ3	Đang thực hiện.

Chương 6. Kết luận và hướng phát triển

6.1 KẾT LUẬN

Tiểu luận đặt câu hỏi "grounded hay guessed?" cho SemLoc [1], phương pháp định vị lỗi ngữ nghĩa tuyên bố neo lập luận của LLM vào bằng chứng thực thi, và so nó với một cách làm đơn giản: chạy kiểm thử, đưa kết quả thất bại cho LLM. Đến thời điểm báo cáo này, nhóm rút ra hai kết luận. Thứ nhất, kết quả công bố tính lại được từ nhật ký, nhưng dòng hạng 1 của SemLoc — theo một lựa chọn thiết kế không có trong bài báo — luôn là dòng LLM tự đoán, và bước vá thử không đổi thứ hạng nào; cách chấm điểm và các nhãn của bộ dữ liệu cũng làm thay đổi đáng kể các con số. Thứ hai, một prompt duy nhất có kiểm thử thất bại thắng quy trình nhiều bước ở Acc@1 trên cả bộ nhãn gốc lẫn bộ đã sửa, và rẻ hơn khoảng bảy lần. Các kết quả này đến từ một mô hình chính và cần được kiểm tra thêm ở các phần sau.

6.2 HƯỚNG PHÁT TRIỂN

Kế hoạch đến bài nộp cuối. Đến báo cáo lần 3: rà soát toàn bộ nhãn của bộ dữ liệu (kèm duyệt độc lập); thí nghiệm loại bỏ để biết thông tin nào trong prompt tạo ra khác biệt; các cách bổ sung (CovGate, biểu quyết) và công cụ phân tích tĩnh; RQ3 trên lỗi thật, mã đổi tên và các họ mô hình; đóng góp, triết lý khoa học và các mối đe dọa. Sau đó: phân tích thất bại, phương pháp mới của nhóm và bài báo tiếng Anh theo chuẩn arXiv.

Về nghiên cứu, bốn hướng mở ra từ kết quả: (i) định vị lỗi cấp dự án — thu hẹp từ cả kho mã xuống một hàm trước khi hỏi LLM, phần khó nhất trên dự án lớn mà tiểu luận chưa đánh giá; (ii) một thí nghiệm giấu tên kiểm thử để đo phần thông tin mà tên kiểm thử "nói hộ"; (iii) bộ kiểm thử chặt hơn (ví dụ sinh thêm kiểm thử phân biệt các bản sửa) để kiểm chứng vị trí lỗi bằng thực thi có ý nghĩa; (iv) một chuẩn đánh giá trên lỗi thật của nhiều dự án, trong đó "một prompt có kiểm thử thất bại" là mốc so sánh bắt buộc.

Tài liệu tham khảo

[1] Z. Yang, H. Zhu, Q. Zhang, R. Gupta, and A. Kundu, “Semloc: Structured grounding of free-form llm reasoning for fault localization,” 2026. [Online]. Available: https://arxiv.org/abs/2603.29109

[2] H. Yang, Y. Peng, and Z. Ding, “Effiholmes: Differential profiling-guided repository level time inefficiency fix localization,” 2026. [Online]. Available: https://arxiv.org/abs/2608.03558

[3] M. Medeiros, U. Kulesza, C. Treude, D. Lucena, R. Gomes, R. Coelho, E. Adachi, and R. Bonifacio, “Integrating crash report mining and llms for bug localization and repair: An industrial report,” 2026. [Online]. Available: https://arxiv.org/abs/2609.04483

[4] K. Beck, *Test-Driven Development: By Example*. Addison-Wesley, 2003.

[5] Stack Overflow, “2025 developer survey,” 2025. [Online]. Available: https://survey.stackoverflow.co/2025/

[6] JetBrains, “The state of developer ecosystem 2025,” 2025. [Online]. Available: https://blog.jetbrains.com/research/2025/10/state-of-developer-ecosystem-2025/

[7] T. Xiao, C. Treude, H. Hata, and K. Matsumoto, “DevGPT: Studying developer-ChatGPT conversations,” in *Proceedings of the 21st International Conference on Mining Software Repositories (MSR), Mining Challenge*, 2024. [Online]. Available: https://arxiv.org/abs/2309.03914

[8] Y. Wu, Z. Li, J. M. Zhang, M. Papadakis, M. Harman, and Y. Liu, “Large language models in fault localisation,” 2023. [Online]. Available: https://arxiv.org/abs/2308.15276

[9] J. A. Jones, M. J. Harrold, and J. Stasko, “Visualization of test information to assist fault localization,” in *Proceedings of the 24th international conference on*

Software engineering - ICSE '02, ser. ICSE '02. ACM Press, 2002, p. 467. [Online]. Available: http://dx.doi.org/10.1145/581396.581397

[10] R. Abreu, P. Zoeteweij, R. Golsteijn, and A. J. van Gemund, "A practical evaluation of spectrum-based fault localization," *Journal of Systems and Software*, vol. 82, no. 11, p. 1780–1792, Nov. 2009. [Online]. Available: http://dx.doi.org/10.1016/j.jss.2009.06.035

[11] W. E. Wong, R. Gao, Y. Li, R. Abreu, and F. Wotawa, "A survey on software fault localization," *IEEE Transactions on Software Engineering*, vol. 42, no. 8, p. 707–740, Aug. 2016. [Online]. Available: http://dx.doi.org/10.1109/TSE.2016.2521368

[12] R. Widyasari, S. Q. Sim, C. Lok, H. Qi, J. Phan, Q. Tay, C. Tan, F. Wee, J. E. Tan, Y. Yieh, B. Goh, F. Thung, H. J. Kang, T. Hoang, D. Lo, and E. L. Ouh, "Bugsinpy: a database of existing bugs in python programs to enable controlled testing and debugging studies," in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE '20. ACM, Nov. 2020, p. 1556–1560. [Online]. Available: http://dx.doi.org/10.1145/3368089.3417943

[13] W. E. Wong, V. Debroy, R. Gao, and Y. Li, "The dstar method for effective software fault localization," *IEEE Transactions on Reliability*, vol. 63, no. 1, p. 290–308, Mar. 2014. [Online]. Available: http://dx.doi.org/10.1109/TR.2013.2285319

[14] H. A. de Souza, M. L. Chaim, and F. Kon, "Spectrum-based software fault localization: A survey of techniques, advances, and challenges," 2016. [Online]. Available: https://arxiv.org/abs/1607.04347

[15] M. Papadakis and Y. Le Traon, "Metallaxis-fl: mutation-based fault localization," *Software Testing, Verification and Reliability*, vol. 25, no. 5-7, p. 605–628, Sep. 2013. [Online]. Available: http://dx.doi.org/10.1002/stvr.1509

[16] S. Moon, Y. Kim, M. Kim, and S. Yoo, "Ask the mutants: Mutating faulty

programs for fault localization," in *2014 IEEE Seventh International Conference on Software Testing, Verification and Validation*. IEEE, Mar. 2014, p. 153–162. [Online]. Available: http://dx.doi.org/10.1109/ICST.2014.28

[17] A. Zeller and R. Hildebrandt, "Simplifying and isolating failure-inducing input," *IEEE Transactions on Software Engineering*, vol. 28, no. 2, p. 183–200, Feb. 2002. [Online]. Available: http://dx.doi.org/10.1109/32.988498

[18] A. Zeller, "Isolating cause-effect chains from computer programs," *ACM SIGSOFT Software Engineering Notes*, vol. 27, no. 6, p. 1–10, Nov. 2002. [Online]. Available: http://dx.doi.org/10.1145/605466.605468

[19] M. D. Ernst, J. H. Perkins, P. J. Guo, S. McCamant, C. Pacheco, M. S. Tschantz, and C. Xiao, "The daikon system for dynamic detection of likely invariants," *Science of Computer Programming*, vol. 69, no. 1-3, p. 35–45, Dec. 2007. [Online]. Available: http://dx.doi.org/10.1016/j.scico.2007.01.015

[20] S. Kang, G. An, and S. Yoo, "A quantitative and qualitative evaluation of llm-based explainable fault localization," 2023. [Online]. Available: https://arxiv.org/abs/2308.05487

[21] Y. Qin, S. Wang, Y. Lou, J. Dong, K. Wang, X. Li, and X. Mao, "Agentfl: Scaling llm-based fault localization to project-level context," 2024. [Online]. Available: https://arxiv.org/abs/2403.16362

[22] C. Xu, Z. Liu, X. Ren, G. Zhang, M. Liang, and D. Lo, "Flexfl: Flexible and effective fault localization with open-source large language models," 2024. [Online]. Available: https://arxiv.org/abs/2411.10714

[23] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based software engineering agents," 2024. [Online]. Available: https://arxiv.org/abs/2407.01489

[24] C. Parnin and A. Orso, "Are automated debugging techniques actually helping

programmers?" in *Proceedings of the 2011 International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2011, pp. 199–209.

[25] P. S. Kochhar, X. Xia, D. Lo, and S. Li, "Practitioners' expectations on automated fault localization," in *Proceedings of the 25th International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2016, pp. 165–176.

[26] R. Just, D. Jalali, and M. D. Ernst, "Defects4j: a database of existing faults to enable controlled testing studies for java programs," in *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, ser. ISSTA '14. ACM, 2014, p. 437–440. [Online]. Available: http://dx.doi.org/10.1145/2610384.2628055

[27] D. Lin, J. Koppel, A. Chen, and A. Solar-Lezama, "Quixbugs: a multi-lingual program repair benchmark set based on the quixey challenge," in *Proceedings Companion of the 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity*, ser. SPLASH '17. ACM, Oct. 2017, p. 55–56. [Online]. Available: http://dx.doi.org/10.1145/3135932.3135941

[28] S. Pearson, J. Campos, R. Just, G. Fraser, R. Abreu, M. D. Ernst, D. Pang, and B. Keller, "Evaluating and improving fault localization," in *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. IEEE, May 2017, p. 609–620. [Online]. Available: http://dx.doi.org/10.1109/ICSE.2017.62

[29] C. G. Northcutt, A. Athalye, and J. Mueller, "Pervasive label errors in test sets destabilize machine learning benchmarks," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021. [Online]. Available: https://arxiv.org/abs/2103.14749

[30] Z. Qi, F. Long, S. Achour, and M. Rinard, "An analysis of patch plausibility and correctness for generate-and-validate patch generation systems," in *Proceedings of the 2015 International Symposium on Software Testing and Analysis (ISSTA)*.

ACM, 2015, pp. 24–36.

[31] E. K. Smith, E. T. Barr, C. Le Goues, and Y. Brun, “Is the cure worse than the disease? overfitting in automated program repair,” in *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. ACM, 2015, pp. 532–543.

[32] J. Sallou, T. Durieux, and A. Panichella, “Breaking the silence: the threats of using llms in software engineering,” 2023. [Online]. Available: https: //arxiv.org/abs/2312.08055

[33] D. Ramos, C. Mamede, K. Jain, P. Canelas, C. Gamboa, and C. L. Goues, “Are large language models memorizing bug benchmarks?” 2024. [Online]. Available: https://arxiv.org/abs/2411.13323

[34] C. Xu, S. Guan, D. Greene, and M.-T. Kechadi, “Benchmark data contamination of large language models: A survey,” 2024. [Online]. Available: https://arxiv.org/abs/2406.04244